ISM3232 — Week 14 Lab

Python + SQL Integration — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 14 · Unit 4 · Capstone Build

Contents

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 14 Lab — Python + SQL Integration
Unit	Unit 4 · Capstone Build
Class length	Full class period (75 minutes)
Format	Live code-along, ending with the full submission ritual
Prerequisites	Week 13: approved `PROPOSAL.md`, `schema.sql`, `sqlite3` shell fluency
Student-facing lab page	Week 14 In-Class Lab — Module 7C, "Python + SQL Integration"
Parts covered	Part 1 (`database.py` setup) – Part 5 (test script + ritual)
Submission	3 screenshots, GitHub URL, Canvas, completion credit

The lab page states two rules up front, applying to *every single line* of today's code, and both deserve to be treated as absolute, not stylistic: **Rule 1 — always use `?` placeholders, never f-strings or string concatenation, to build SQL. Rule 2 — every function accepts an optional `db_file` parameter, defaulting to `DB_FILE`, so tests can use an isolated temporary database.** Rule 1 is a genuine security discipline (SQL injection prevention), not a style preference — worth demonstrating the actual vulnerability, not just stating the rule. Rule 2 is what makes Part 4's `pytest` tests possible at all without corrupting the real development database. Today's `database.py` becomes the actual data layer the capstone's Streamlit interface (Week 15) reads from and writes to — this is genuinely load-bearing code, not a standalone exercise.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Connect to a SQLite database from Python with `sqlite3.connect()`, using a `with` block for automatic connection handling.
2. Use `?` placeholders for every value inserted into a SQL query, and explain concretely why string formatting/concatenation into SQL is a security risk, not just a style issue.
3. Use `row_factory = sqlite3.Row` to retrieve query results as dictionary-like rows instead of plain tuples.
4. Design every database function to accept an optional `db_file` parameter, enabling isolated testing.

5. Write `pytest` tests using the `tmp_path` fixture to test against a fresh, isolated database file per test.

3 Before Class — Setup Checklist

- ☐ Rehearse the SQL injection demonstration (see Part 2's expanded explanation) before class — showing the actual vulnerability, not just asserting it exists, is worth the extra few minutes and is one of the most memorable security lessons this course offers.
- ☐ Confirm your own `database.py` runs cleanly end to end before class — this lab has five interdependent functions building on each other, and a broken function 2 cascades into confusing failures in functions 3–5 if not caught early.
- ☐ Note explicitly for yourself before demonstrating Part 5's `test_script.py`: `get_all_records()` orders by `id DESC` (most recent first), so `records[0]` is the **most recently added** record, not the first one — this genuinely surprises people the first time and is worth flagging deliberately rather than let it pass as a throwaway detail (see Part 5's walkthrough).

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module07_final_project` venv from Week 13
- Students: their approved `PROPOSAL.md` and `schema.sql` from Week 13 (today builds the general pattern using the lab's own `requests` table; students should be ready to adapt it to their own capstone schema afterward)

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: "the two absolute rules"	4
0:04–0:14	Part 1 — Set up `database.py`	10
0:14–0:28	Part 2 — `add_record` and `get_all_records`	14
0:28–0:38	Part 3 — `update_status` and `get_status_report`	10
0:38–0:58	Part 4 — Five pytest tests with `tmp_path`	20
0:58–1:10	Part 5 — Test script and ritual	12
1:10–1:15	Wrap-up, submission checklist	5

Part 4 receives the most time of any single part — the `tmp_path` fixture and the whole "isolated test database" pattern is genuinely new machinery this lab introduces, worth unhurried treatment, since it directly depends on Rule 2 being correctly applied throughout Parts 1–3.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

> *"Two rules today, and they apply to every single line of code you write, no exceptions. One: SQL values always go in through `?` placeholders — never build a query string with an f-string or plus-concatenation. Two: every function takes an optional `db_file` parameter, defaulting to your real database, so tests can point at a throwaway one instead. Rule one is a real security discipline. Rule two is what makes today's tests possible at all. By the end of today, this file — `database.py` — is the actual data layer your capstone's interface reads from and writes to next week."*

6.2 Part 1 — Set Up `database.py` (0:04–0:14, 10 min)

Teaching goal: The connection pattern (`with sqlite3.connect(...)`) and the first of five required functions, `create_table()`.

Say to the class:

> *"First function: create the table if it doesn't already exist. Notice the `with` block — same pattern as every file-handling exercise since Module 12, now managing a database connection instead of a file."*

Live-code this:

```
cd ~/ism3232/module07_final_project
source .venv/bin/activate
mkdir -p data tests
touch database.py tests/__init__.py tests/test_database.py
code database.py
```

```python
import sqlite3

DB_FILE = 'data/requests.db'

def create_table(db_file=DB_FILE):
    """Create the main table if it does not already exist."""
    with sqlite3.connect(db_file) as conn:
        conn.execute('''
            CREATE TABLE IF NOT EXISTS requests (
                id        INTEGER PRIMARY KEY AUTOINCREMENT,
                requester TEXT    NOT NULL,
```

```
                category  TEXT    NOT NULL,
                amount    REAL,
                status    TEXT    DEFAULT 'Pending',
                notes     TEXT
            )
        ''')
```

Line-by-line explanation:

- `import sqlite3` — part of Python's standard library, no installation needed — the same underlying engine as the `sqlite3` shell command used all last week, now driven from Python instead of the command line directly.
- `DB_FILE = 'data/requests.db'` — a module-level constant naming the **default** database file — say explicitly, matching Rule 2's spirit: this is the "real" development database path, used automatically unless a function is told otherwise.
- `def create_table(db_file=DB_FILE):` — **this is Rule 2, in its very first application.** `db_file=DB_FILE` is a **default parameter value** — calling `create_table()` with no arguments uses `DB_FILE`; calling `create_table(some_other_path)` uses that instead. Say explicitly: every one of today's five functions repeats this exact pattern, and it's worth internalizing now, on the simplest function, before it appears four more times.
- `with sqlite3.connect(db_file) as conn:` — opens a connection to the named database file (creating the file itself if it doesn't yet exist) — the `with` block guarantees the connection closes properly even if something goes wrong inside it, exactly the same safety rationale as file-handling `with` blocks.
- `conn.execute('''...''')` — runs a SQL statement through the connection; note the **triple-quoted string**, new here specifically because this SQL statement spans multiple lines — triple quotes allow a string literal to contain real line breaks, which single or double quotes don't.
- `CREATE TABLE IF NOT EXISTS requests (...)` — **`IF NOT EXISTS` is new relative to Week 13's shell version** — say explicitly why it matters here specifically: this function might be called many times across a script's life (every time the app starts, say), and without `IF NOT EXISTS`, a second call would raise an error complaining the table already exists; with it, calling `create_table()` repeatedly is safe and does nothing on the second and later calls.
- The column definitions themselves — identical vocabulary to Week 13 (`INTEGER PRIMARY KEY AUTOINCREMENT`, `TEXT NOT NULL`, `DEFAULT 'Pending'`), with one new column, `notes TEXT`, added for this lab's slightly richer example.

Common student mistakes to watch for:

- Forgetting `mkdir -p data` before running anything that tries to create `data/requests.db` — `sqlite3.connect()` can fail if the parent directory doesn't exist; a quick `ls data/` check confirms the folder is present first.
- Using single quotes for the multi-line `CREATE TABLE` string instead of triple quotes — produces

a `SyntaxError` from the embedded, unescaped line breaks; a good moment to point at the triple-quote requirement explicitly if it comes up.

Check for understanding: “If `create_table()` were called twice in a row, what would happen the second time, and why is that safe?” (Nothing happens the second time — `IF NOT EXISTS` means the statement is a no-op if the table is already there; get a student to state this is *deliberately* safe, not accidentally harmless.)

6.3 Part 2 — `add_record` and `get_all_records` (0:14–0:28, 14 min)

Teaching goal: The two most important functions in this file — writing data safely with `?` placeholders, and reading it back as usable dictionaries with `row_factory`.

Say to the class:

> *"This is where Rule 1 matters most. I'm going to show you what goes wrong if you don't follow it — not just tell you."*

Live-code this, added to `database.py`:

```
def add_record(requester, category, amount, notes='', db_file=DB_FILE):
    """Insert a new record. Uses ? placeholders -- never f-strings."""
    with sqlite3.connect(db_file) as conn:
        conn.execute(
            'INSERT INTO requests (requester, category, amount, notes) VALUES (?,?,?,?)',
            (requester, category, amount, notes)
        )
```

Line-by-line explanation — this is Rule 1, worth the most careful explanation of the whole lab:

- `'INSERT INTO requests (requester, category, amount, notes) VALUES (?,?,?,?)'` — the SQL string itself contains `?` **placeholders**, not the actual values.
- `(requester, category, amount, notes)` — a **second, separate argument** to `.execute()`: a tuple of the actual values, matched positionally to each `?` in order. Say explicitly: `sqlite3` fills in each placeholder itself, safely — it's not simply substituting text into the string the way an f-string would.

Now show, deliberately, what a broken (f-string) version would look like — do not run this against a real table, demonstrate the *idea* rather than the exploit itself:

```
# NEVER DO THIS -- shown only to explain why Rule 1 exists
def add_record_UNSAFE(requester, category, amount):
    query = f"INSERT INTO requests (requester, category, amount) VALUES ('{requester}', '{ca
    # if `requester` were:  Taylor', 'Travel', 999); DROP TABLE requests; --
    # the f-string would build a COMPLETELY different, malicious SQL
    # statement -- not because of a bug in this function, but because
    # untrusted text was allowed to become part of the SQL itself.
```

Explain, without running the destructive example live: "If `requester` ever came from a user-facing form — which, in your capstone's Streamlit app next week, it genuinely will — a malicious or even just unusually-formatted input (a name containing a stray apostrophe, even innocently) could corrupt or hijack the query. `?` placeholders make this structurally impossible: the *value*

and the *SQL structure* are always kept completely separate, no matter what text a value contains. This is called **SQL injection**, and it's one of the most common real-world security vulnerabilities in software history — Rule 1 exists because this genuinely matters, not as an academic exercise."

Now `get_all_records`:

```
def get_all_records(db_file=DB_FILE):
    """Return all records as a list of dicts."""
    with sqlite3.connect(db_file) as conn:
        conn.row_factory = sqlite3.Row
        cur = conn.execute('SELECT * FROM requests ORDER BY id DESC')
        return [dict(row) for row in cur.fetchall()]
```

Line-by-line explanation:

- `conn.row_factory = sqlite3.Row` — **without this line, query results come back as plain tuples** (`(1, 'Taylor', 'Travel', 1200.0, 'Pending', '')`), accessible only by position (`row[0]`, `row[1]`, …) — which is workable but far less readable than named access. Setting `row_factory` to `sqlite3.Row` changes this: each row can then be accessed *both* by position and by column name.
- `cur = conn.execute('SELECT * FROM requests ORDER BY id DESC')` — **note `ORDER BY id DESC`, deliberately** — say explicitly, and flag this clearly since it matters later: this means the **most recently added** record comes first in the results, not the first one ever added. Worth writing on the board now, since Part 5 revisits this exact detail as a genuine "read the code carefully" moment.
- `[dict(row) for row in cur.fetchall()]` — `cur.fetchall()` retrieves every matching row; the list comprehension wraps each `sqlite3.Row` object in `dict(...)`, converting it into an ordinary Python dictionary — say explicitly why this final conversion matters: a plain `sqlite3.Row` object supports name-based access already, but converting to a real `dict` makes the returned data fully ordinary and familiar — usable with every dictionary technique from Weeks 6, 11, and 12, with no special `sqlite3`-specific knowledge required by any code that calls this function.

Now, test manually in a Python shell, exactly as the lab page specifies:

```
python3
>>> from database import create_table, add_record, get_all_records
>>> create_table()
>>> add_record('Taylor', 'Travel', 1200)
>>> records = get_all_records()
>>> print(records[0])
>>> exit()
```

Verified output:

```
{'id': 1, 'requester': 'Taylor', 'category': 'Travel', 'amount': 1200.0, 'status': 'Pending'
```

Point out explicitly: this printed as a genuine Python dictionary, with real key names — directly usable with `records[0]['requester']`, exactly like every dictionary since Week 6, with zero special SQL-aware syntax needed by anything downstream of `get_all_records()`.

Common student mistakes to watch for:

- Forgetting `conn.row_factory = sqlite3.Row` — results still work, but come back as plain tuples; `records[0]['requester']` would then fail with `TypeError: tuple indices must be integers or slices, not str`, a good, specific error confirming exactly what's missing.
- Passing values directly into the SQL string instead of via the placeholder tuple, even without going as far as an f-string (e.g., `conn.execute(f'... VALUES ({amount})')` for just one value) — worth catching even a partial violation of Rule 1, not just a fully f-string version.

Check for understanding: "If a user's `requester` name were `O'Brien` — containing an apostrophe — would `add_record` with `?` placeholders handle it correctly?" (Yes, without any special handling needed — this is exactly the kind of input that would silently break a naive f-string-built query (an unescaped apostrophe would prematurely end a quoted SQL string literal), while `?` placeholders handle it completely correctly and automatically, with no extra code required. A good, concrete, non-malicious example of why Rule 1 matters even without imagining an attacker.)

6.4 Part 3 — `update_status` and `get_status_report` (0:28–0:38, 10 min)

Teaching goal: The remaining two functions — completing the full CRUD-and-report pattern (create, read, update, aggregate-report) with the same two rules applied throughout.

Say to the class:

> *"Two more functions, same two rules, no new ideas — just applying what you already have to `UPDATE` and `GROUP BY`."*

Live-code this, added to `database.py`:

```
def update_status(record_id, new_status, db_file=DB_FILE):
    """Update status of a record by ID."""
    with sqlite3.connect(db_file) as conn:
        conn.execute('UPDATE requests SET status=? WHERE id=?', (new_status, record_id))

def get_status_report(db_file=DB_FILE):
    """Return count and total grouped by status."""
    with sqlite3.connect(db_file) as conn:
        cur = conn.execute('SELECT status, COUNT(*) as count, SUM(amount) as total FROM requ
        return cur.fetchall()
```

Line-by-line explanation:

- `'UPDATE requests SET status=? WHERE id=?', (new_status, record_id)` — **two placeholders this time, matched positionally in order**: the first `?` gets `new_status`, the second gets `record_id`, exactly matching their order in the tuple. Say explicitly: get this order wrong (swap the tuple to `(record_id, new_status)`) and the query would try to set `status` to a number and filter `WHERE id=` a status string — worth demonstrating live if a student makes this mistake, since the resulting error or silently-wrong behavior is genuinely instructive.
- `get_status_report` — **note this function does NOT use `row_factory/dict()`** — it returns `cur.fetchall()` directly, as plain tuples, unlike `get_all_records()`. Say explicitly why this is a deliberate, meaningful difference, not an oversight: each row here is `(status, count, total)` — three positional values with clear, fixed meaning by position, genuinely fine to access as `row[0]`, `row[1]`, `row[2]` without needing named dictionary access. This is worth stating as a real design choice: **not every query result needs the dictionary treatment** — use it when named access genuinely improves clarity (a record with many fields), skip it when a short, fixed-position tuple is already clear enough.

Verify by hand, continuing the REPL session from Part 2 (or running a fresh script):

```
update_status(1, 'Approved')
report = get_status_report()
```

```
print(report)
```

Verified output:

```
[('Approved', 1, 1200.0)]
```

Common student mistakes to watch for:

- Swapping the placeholder order in `update_status`, discussed above.
- Expecting `get_status_report()`'s rows to support `row['status']`-style access, out of habit from `get_all_records()` — they don't, since `row_factory` wasn't set for this connection; `row[0]` is the correct access pattern here, worth an explicit contrast with Part 2's function.

Check for understanding: "Why doesn't `get_status_report()` need `row_factory = sqlite3.Row` the way `get_all_records()` does?" (Because its result rows are short, fixed-shape tuples (status, count, total) whose meaning is clear from position alone — dictionary-style access would add ceremony without adding clarity here, unlike a five-or-six-column full record row where remembering "is `amount` the third or fourth position" genuinely gets error-prone.)

6.5 Part 4 — Five pytest Tests with `tmp_path` (0:38–0:58, 20 min)

Teaching goal: `tmp_path` — a `pytest` fixture creating a fresh, isolated temporary folder for every single test — and Rule 2's actual payoff: every function's `db_file` parameter exists specifically to make this possible.

Say to the class:

> *"This is the entire reason every function today takes a `db_file` parameter. Watch: every test gets its* own*, completely separate, temporary database file — nothing here ever touches your real `data/requests.db`, and no test can accidentally see data left over from a different test."*

Live-code this:

```
# tests/test_database.py
import pytest
from database import create_table, add_record, get_all_records, update_status, get_status_re

def test_add_and_retrieve(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('Taylor', 'Travel', 1200, db_file=db)
    records = get_all_records(db_file=db)
    assert len(records) == 1
    assert records[0]['requester'] == 'Taylor'
    assert records[0]['amount'] == 1200

def test_default_status_is_pending(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('Jordan', 'Equipment', 450, db_file=db)
    records = get_all_records(db_file=db)
    assert records[0]['status'] == 'Pending'

def test_update_status(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('Morgan', 'Software', 3500, db_file=db)
    records = get_all_records(db_file=db)
    update_status(records[0]['id'], 'Approved', db_file=db)
    updated = get_all_records(db_file=db)
    assert updated[0]['status'] == 'Approved'
```

```python
def test_multiple_records(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('A', 'T', 500, db_file=db)
    add_record('B', 'T', 750, db_file=db)
    records = get_all_records(db_file=db)
    assert len(records) == 2

def test_status_report_groups_correctly(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('A', 'T', 500, db_file=db)
    add_record('B', 'T', 750, db_file=db)
    records = get_all_records(db_file=db)
    update_status(records[0]['id'], 'Approved', db_file=db)
    report = get_status_report(db_file=db)
    statuses = [row[0] for row in report]
    assert 'Pending' in statuses
    assert 'Approved' in statuses
```

Line-by-line explanation:

- `def test_add_and_retrieve(tmp_path):` — **`tmp_path` is a `pytest` fixture** — say explicitly what that means precisely: a special parameter name that `pytest` recognizes and automatically supplies a value for, without the test ever calling anything to request it. `pytest` creates a brand-new, empty temporary directory on disk, unique to *this specific test run*, and hands its path in as `tmp_path`.
- `db = tmp_path / 'test.db'` — note the `/` here is **not division** — `tmp_path` is a `pathlib.Path` object (not a plain string), and `Path` objects overload `/` specifically to mean "join a path segment," a genuinely elegant piece of Python worth a brief explicit mention: this line builds a full file path like `/tmp/pytest-.../test0/test.db`, combining the fixture's unique temp folder with a filename.
- `create_table(db)`, `add_record(..., db_file=db)`, `get_all_records(db_file=db)` — **every single call passes `db` explicitly** — this is Rule 2's entire payoff, worth stating plainly: because every function accepts `db_file` as a parameter instead of hardcoding `DB_FILE` internally, tests can redirect every operation to this test's own private, temporary file, completely isolated from the real `data/requests.db` and from every *other* test's own `tmp_path`.
- `test_status_report_groups_correctly` — the richest test, combining `add_record`, `get_all_records`, `update_status`, and `get_status_report` in one sequence — say explicitly, this is closer to a genuine end-to-end integration test than a narrow unit test, and that's a deliberate, reasonable choice for testing a small, tightly-coupled data layer like this.

Run it:

```
pytest -v
```

Verified output — all five pass:

```
tests/test_database.py::test_add_and_retrieve PASSED
tests/test_database.py::test_default_status_is_pending PASSED
tests/test_database.py::test_update_status PASSED
tests/test_database.py::test_multiple_records PASSED
tests/test_database.py::test_status_report_groups_correctly PASSED
5 passed
```

A genuinely worthwhile live demo, if time allows: temporarily hardcode `db_file=DB_FILE` inside `add_record` (ignoring the parameter, violating Rule 2 on purpose) and re-run the tests — watch them either fail or, worse, silently start writing to the *real* development database instead of the isolated temp one. This is worth showing concretely, since "why does Rule 2 matter" can otherwise feel abstract until seen breaking.

Common student mistakes to watch for:

- Creating a *new* `tmp_path / 'test.db'` inside a test but forgetting to pass `db_file=db` to one of the four function calls, silently defaulting that one call back to the real `DB_FILE` — a subtle, easy-to-miss mistake; walk the room checking every single function call within each test passes `db_file` explicitly.
- Assuming `tmp_path` is shared *across* tests in the same file — it isn't; each test function gets its own fresh, separate `tmp_path`, which is precisely the isolation guarantee this whole pattern provides. A student who expects data from `test_add_and_retrieve` to still be present in `test_default_status_is_pending` has misunderstood the fixture's actual scope.

Check for understanding: "If Rule 2 had never been applied — if every function hardcoded `DB_FILE` internally with no parameter — could these five tests still pass?" (Not safely — they might *appear* to pass individually, but would all be reading and writing the same real database file, meaning tests could interfere with each other's data, and running the test suite would pollute the actual development database with test records. A good check that "the tests happen to produce green checkmarks" and "the tests are actually correctly isolated" are different, and only the second is what this lab is really after.)

6.6 Part 5 — Test Script and Ritual (0:58–1:10, 12 min)

Teaching goal: A standalone script exercising the real `database.py` against the actual development database — and a genuine "read the code, don't assume" moment around `get_all_records()`'s ordering.

Say to the class:

> *"One more script, this time against your real database, not a temp one — showing everything working together end to end. And I want you to predict something before we run it."*

Live-code this:

```
# test_script.py
from database import create_table, add_record, get_all_records, update_status, get_status_re

create_table()
add_record('Taylor', 'Travel', 1200)
add_record('Jordan', 'Equipment', 450)
add_record('Morgan', 'Software', 3500)

records = get_all_records()
print(f'Records: {len(records)}')

update_status(records[0]['id'], 'Approved')

report = get_status_report()
print('\nStatus report:')
for row in report:
    print(f'  {row[0]}: {row[1]} records, ${row[2]:,.2f}')
```

Before running it, ask the room explicitly: "Three records added — Taylor, Jordan, Morgan, in that order. `update_status(records[0]['id'], 'Approved')` approves *which one*? Predict it, based on Part 2's `ORDER BY id DESC` detail."

Run it:

```
python3 test_script.py
```

Verified output:

```
Records: 3

Status report:
  Approved: 1 records, $3,500.00
  Pending: 2 records, $1,650.00
```

Confirm explicitly, since this is worth landing deliberately: "$3,500 approved — that's **Morgan**, the *last* record added, not Taylor, the first. `records[0]` is the *first item in the list returned by `get_all_records()`*, and that function orders by `id DESC` — most recent first. If you assumed `records[0]` meant 'the first request I added,' this output would look wrong. It isn't wrong — the code did exactly what it says; the assumption about what `records[0]` means was the actual gap." This is worth treating as a genuine highlight of the lab, not an aside: **it's a real example of code being completely correct while still producing a surprising result, purely because of an unstated assumption about ordering** — exactly the kind of subtle bug source that a quick test (Part 4) or a careful read (right now) catches, and an assumption-driven skim would miss entirely.

Now the full ritual:

```
ruff format . && ruff check . && pytest -v
git add . && git commit -m 'lab 14: Python SQL integration' && git push
```

Common student mistakes to watch for:

- Running `test_script.py` multiple times without clearing `data/requests.db` between runs — since `add_record` always inserts new rows (never resets the table), running it twice produces six records total, not three; if a student's `Records: 3` doesn't match, checking whether this is a repeat run is the first thing to verify.
- Assuming the `Status report` line order (`Approved` before `Pending`) reflects something meaningful (like insertion order or urgency) rather than just whatever order `GROUP BY` happened to produce — worth a brief note that `GROUP BY`'s result order isn't guaranteed or semantically meaningful unless an explicit `ORDER BY` is added afterward.

Check for understanding: "If you wanted `test_script.py` to approve the *first-ever* added record (Taylor) instead of the most recent one, what would need to change?" (Either query specifically for the record with the minimum `id` — e.g., `records[-1]` if the list stays DESC-ordered, since the oldest record would then be *last* in the list — or change `get_all_records()`'s `ORDER BY` to `ASC` instead of `DESC`. Getting a student to propose a concrete fix confirms the ordering behavior, and its consequences, genuinely landed.)

7 Wrap-Up (last ~5 minutes)

Review the submission checklist together:

- ☐ `database.py` contains all five functions: `create_table`, `add_record`, `get_all_records`, `update_status`, `get_status_report`
- ☐ Every function uses `?` placeholders for values, never f-strings/concatenation in SQL (Rule 1)
- ☐ Every function accepts an optional `db_file` parameter defaulting to `DB_FILE` (Rule 2)
- ☐ `tests/test_database.py` contains all five tests, all using `tmp_path`, all passing
- ☐ `test_script.py` runs cleanly and shows the status report
- ☐ Git commit made, with a message including "lab 14"
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 15: "Today's five functions are the entire data layer your capstone needs. Next week, Streamlit — a real, clickable web interface, calling exactly these functions (`add_record`, `get_all_records`, `update_status`, `get_status_report`) to let a genuine user interact with your database without touching a terminal or the `sqlite3` shell at all."

8 Appendix A — Full Answer Key (`database.py` + `tests/test_database.py` + `test_script.py`)

```python
# database.py
import sqlite3

DB_FILE = 'data/requests.db'

def create_table(db_file=DB_FILE):
    """Create the main table if it does not already exist."""
    with sqlite3.connect(db_file) as conn:
        conn.execute('''
            CREATE TABLE IF NOT EXISTS requests (
                id        INTEGER PRIMARY KEY AUTOINCREMENT,
                requester TEXT    NOT NULL,
                category  TEXT    NOT NULL,
                amount    REAL,
                status    TEXT    DEFAULT 'Pending',
                notes     TEXT
            )
        ''')

def add_record(requester, category, amount, notes='', db_file=DB_FILE):
```

```
    """Insert a new record into the table."""
    with sqlite3.connect(db_file) as conn:
        conn.execute(
            'INSERT INTO requests (requester, category, amount, notes) VALUES (?,?,?,?)',
            (requester, category, amount, notes)
        )

def get_all_records(db_file=DB_FILE):
    """Return all records as a list of dicts."""
    with sqlite3.connect(db_file) as conn:
        conn.row_factory = sqlite3.Row
        cur = conn.execute('SELECT * FROM requests ORDER BY id DESC')
        return [dict(row) for row in cur.fetchall()]

def update_status(record_id, new_status, db_file=DB_FILE):
    """Update the status of a record by ID."""
    with sqlite3.connect(db_file) as conn:
        conn.execute('UPDATE requests SET status=? WHERE id=?', (new_status, record_id))

def get_status_report(db_file=DB_FILE):
    """Return count and total grouped by status."""
    with sqlite3.connect(db_file) as conn:
        cur = conn.execute('SELECT status, COUNT(*) as count, SUM(amount) as total FROM requ
        return cur.fetchall()
```

```
# tests/test_database.py
import pytest
from database import create_table, add_record, get_all_records, update_status, get_status_re

def test_add_and_retrieve(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('Taylor', 'Travel', 1200, db_file=db)
    records = get_all_records(db_file=db)
    assert len(records) == 1
    assert records[0]['requester'] == 'Taylor'
    assert records[0]['amount'] == 1200

def test_default_status_is_pending(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
```

```
    add_record('Jordan', 'Equipment', 450, db_file=db)
    records = get_all_records(db_file=db)
    assert records[0]['status'] == 'Pending'

def test_update_status(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('Morgan', 'Software', 3500, db_file=db)
    records = get_all_records(db_file=db)
    update_status(records[0]['id'], 'Approved', db_file=db)
    updated = get_all_records(db_file=db)
    assert updated[0]['status'] == 'Approved'

def test_multiple_records(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('A', 'T', 500, db_file=db)
    add_record('B', 'T', 750, db_file=db)
    records = get_all_records(db_file=db)
    assert len(records) == 2

def test_status_report_groups_correctly(tmp_path):
    db = tmp_path / 'test.db'
    create_table(db)
    add_record('A', 'T', 500, db_file=db)
    add_record('B', 'T', 750, db_file=db)
    records = get_all_records(db_file=db)
    update_status(records[0]['id'], 'Approved', db_file=db)
    report = get_status_report(db_file=db)
    statuses = [row[0] for row in report]
    assert 'Pending' in statuses
    assert 'Approved' in statuses
```

```
# test_script.py
from database import create_table, add_record, get_all_records, update_status, get_status_re

create_table()
add_record('Taylor', 'Travel', 1200)
add_record('Jordan', 'Equipment', 450)
add_record('Morgan', 'Software', 3500)
```

```python
records = get_all_records()
print(f'Records: {len(records)}')

update_status(records[0]['id'], 'Approved')

report = get_status_report()
print('\nStatus report:')
for row in report:
    print(f'  {row[0]}: {row[1]} records, ${row[2]:,.2f}')
```

Verified output:

```
Records: 3

Status report:
  Approved: 1 records, $3,500.00
  Pending: 2 records, $1,650.00
```

9 Appendix B — Extra Practice (only if the class finishes early)

Five required functions plus tests fill the full class period at a normal pace. If a section moves unusually fast:

Extra — a `delete_record` function. Have students add `def delete_record(record_id, db_file=DB_FILE): with sqlite3.connect(db_file) as conn: conn.execute('DELETE FROM requests WHERE id=?', (record_id,))` — note the single-element tuple `(record_id,)` needs the trailing comma to actually be a tuple, not just parentheses around one value (a classic, worth-flagging Python syntax trap) — and write a `tmp_path`-based test confirming the record count decreases by one after deletion.

Extra — adapt today's pattern to the student's own capstone schema. Have students copy `database.py`'s five-function shape into a version using their own Week 13 `schema.sql` table(s) and field names, confirming the same `?`-placeholder and `db_file`-parameter rules apply identically regardless of the specific domain — genuinely useful, directly-reusable work toward their actual capstone, not just extra practice for its own sake.